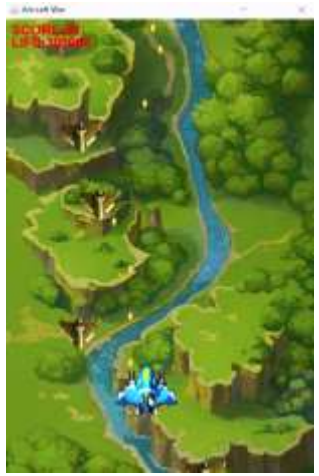
实验二：简单工厂和工厂方法模式

实验与创新实践教育中心 • 计算机与数据技术实验教学部

本学期实验总体安排

初始版本



最终版本

初始版本

游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

01

UML建模
精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机

02

采用**简单工厂模式**
创建道具
采用**工厂方法模式**
创建敌机

03

采用**策略模式**
实现不同飞机的发射弹道
两种火力道具生效

04

采用**数据访问对象模式**
实现得分排行榜
添加**JUnit单元测试**

05

使用**Swing**添加游戏难度选择和排行榜界面
使用**多线程**实现音效控制及火力道具状态恢复

06

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

本学期实验总体安排

实验项目	二	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	系统分析 单例模式	**简单工厂 工厂方法 模式**	策略模式	JUnit 数据访问 对象模式	Swing 多线程	观察者模式 模板模式
时间节点			**中期**			**结项**
提交内容			UML类图 项目代码			结项报告 代码、类图 展示视频

实验课程共 16 个学时，6 个实验项目，总成绩为 30分 。

目录

CONTENTS

01 实验目的

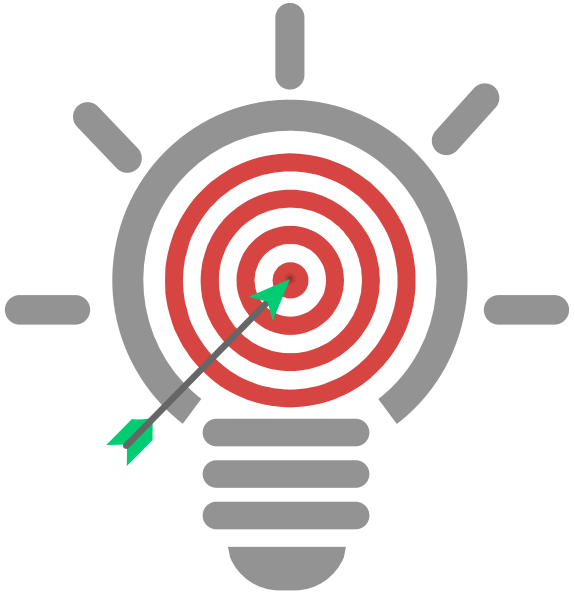
02 实验任务

03 实验原理

04 实验步骤

实验目的

- 深入理解**简单工厂**与**工厂方法模式**的设计意图及结构
- 结合案例，规范绘制两种模式的**UML类图**
- 掌握**重构**方法，通过**代码实现**并应用两种工厂模式



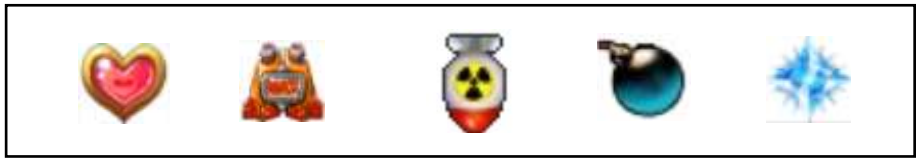
实验任务

1. 深入学习**简单工厂模式**与**工厂方法模式**的设计原理，结合实例完成两种模式的**UML结构图**绘制；
2. 对系统中的敌机、道具创建逻辑进行代码重构：
   - 采用简单工厂实现道具对象的创建
   - 采用工厂方法模式实现敌机对象的创建
3. 完成本次迭代开发清单中的全部任务

实验步骤：道具创建场景分析（1）

道具创建场景分析

游戏中有5种道具：加血、火力、超级火力、炸弹和冰冻道具。



所有道具均在特定敌机被击毁时按预设概率随机掉落生成，生成后以固定速度向屏幕下方移动，英雄机碰撞后自动触发对应效果，不同道具的效果存在差异。

实验步骤：道具创建场景分析（2）

请思考：

1. 掉落道具的职责分给哪个类?

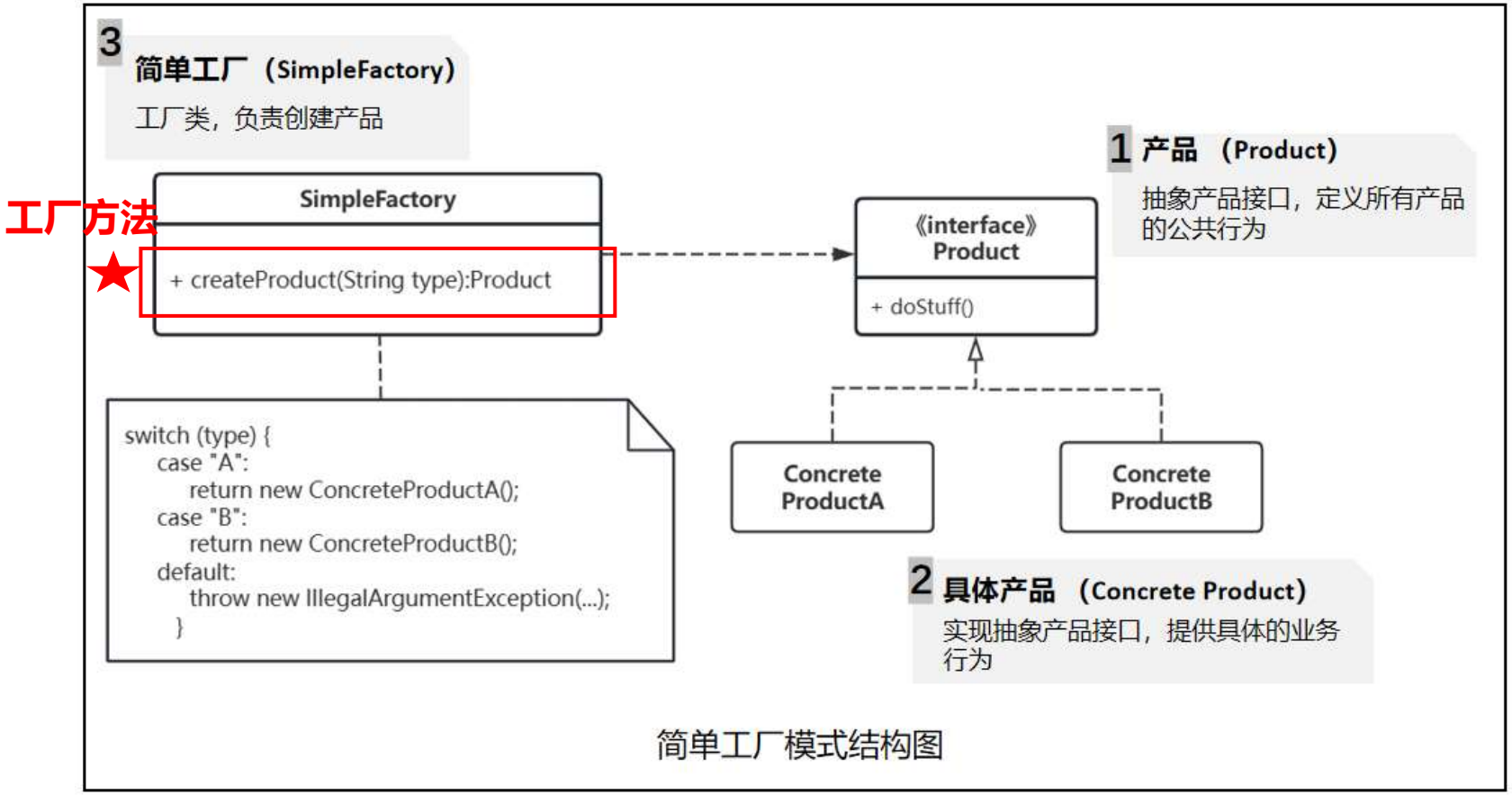
Game类 ✗　　敌机子类 ✗　　敌机父类 ✓

2. 如果增加一种新型道具，需要修改哪些代码?

封装变化 → 将所有对象的创建集中在一个地方管理

实验步骤：简单工厂模式结构图

简单工厂模式（Simple Factory Pattern）属于创建型设计模式，由一个工厂类根据传入的参数，决定创建并返回哪一种产品类实例。



简单工厂模式结构图

实验步骤：简单工厂模式示例代码

① 创建一个接口，充当产品角色；

```java
public interface Product {
    void show();
}
```

② 创建两个实体类，充当具体产品角色；

```java
public class ConcreteProductA implements Product {
    @Override
    public void show() {
        System.out.println("I am product A.");
    }
}

public class ConcreteProductB implements Product {
    @Override
    public void show() {
        System.out.println("I am product B.");
    }
}
```

③ 创建一个简单工厂类，负责创建产品；

```java
public class SimpleFactory {
    public static Product createProduct(String type) {
        switch (type) {
            case "A":
                return new ConcreteProductA();
            case "B":
                return new ConcreteProductB();
            default:
                throw new IllegalArgumentException("Unknown product type! ");
        }
    }
}
```

④ 客户端调用工厂方法获取不同类型的产品

```java
public class SimpleFactoryPatternDemo {
    public static void main(String[] args) {
        Product productA = SimpleFactory.createProduct("A");
        productA.show();
        Product productB = SimpleFactory.createProduct("B");
        productB.show();
    }
}
```

实验步骤：敌机创建场景分析（1）

敌机
创建场景
分析

游戏中，敌机是核心作战单位，目前主要涵盖5种类型：普通、精英、精锐、王牌和Boss 敌机。

不同类型敌机在属性、行为及生成逻辑上存在显著差异，且游戏运行过程中会持续、随机生成各类敌机，生成逻辑分散且复杂。另外，随着游戏版本的迭代，未来极可能引入更多新型敌机，系统必须具备良好的可扩展性。

实验步骤：敌机创建场景分析（2）

请思考：

针对接口编程，而不是针对实现编程！

1. 目前在哪个类创建敌机？如何创建？

```
if (enemyAircrafts.size() < enemyMaxNumber) {
    enemyAircrafts.add(new MobEnemy(
            (int) (Math.random() * (Main.WINDOW_WIDTH - ImageManager.MOB_ENEMY_IMAGE.getWidth())),
            (int) (Math.random() * Main.WINDOW_HEIGHT * 0.05),
            speedX: 0,
            speedY: 10,
            hp: 30
    ));
}
```

Game类

违反单一职责 X

2. 若敌机增加一种属性 “防御力”，需要改动哪些代码？

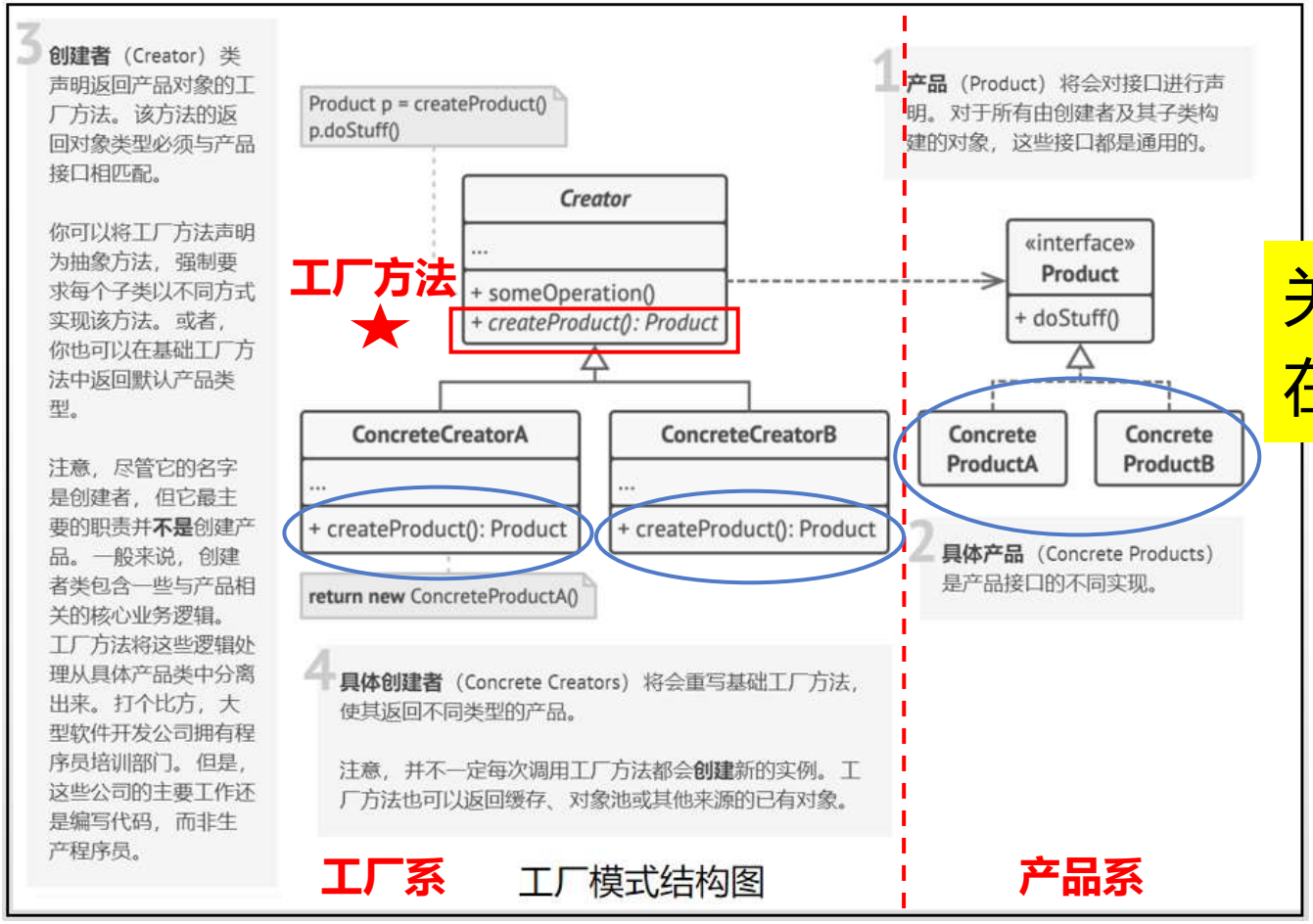
违反开闭原则 X

3. 若增加一种新型敌机，需要改动哪些代码？

违反依赖倒转 X

实验步骤：工厂方法模式结构图

工厂方法模式（Factory Method Pattern）也是一种创建型设计模式， 其在父类中提供一个创建对象的方法， 由子类决定实例化对象的类型。



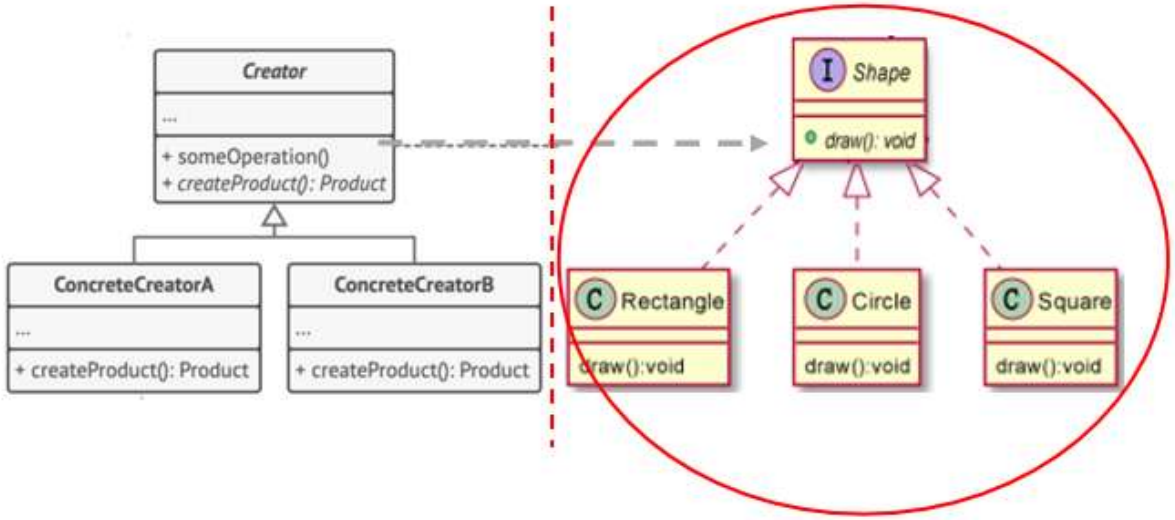
关键代码：创建过程在其子类执行

实验步骤：图形工厂举例（工厂方法模式）

假如我们要开发一个图形工厂，可生产3种类型的图形产品：圆形、长方形、正方形。我们该如何用工厂方法模式实现呢？

实验步骤：图形工厂举例（1）

产品系：创建 Shape 接口和实现该接口的三个图形实体类。



① 创建Shape 接口, 充当产品角色；

```
public interface Shape {
    void draw();
}
```

② 创建三个图形实体类，充当具体产品角色；

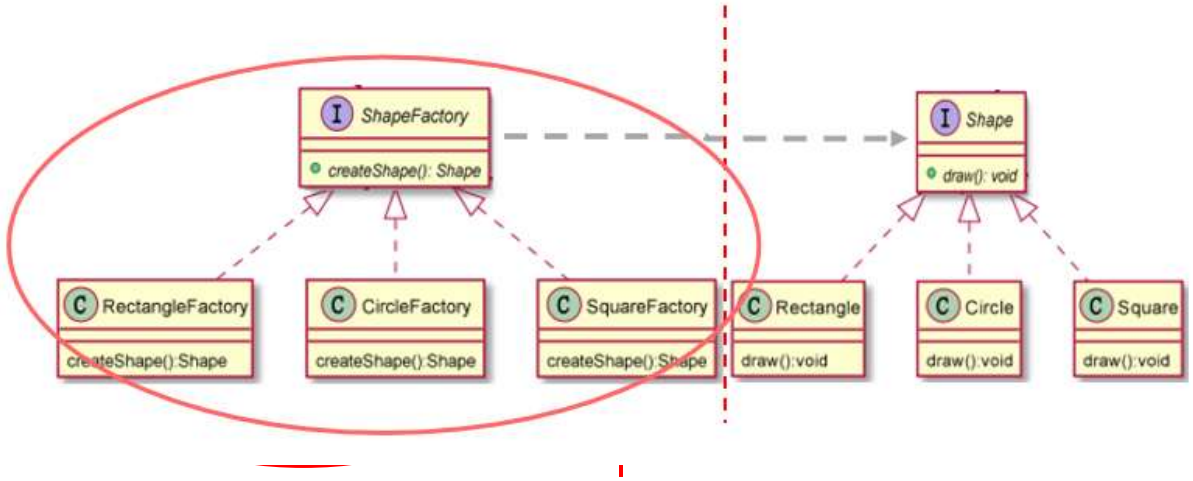
```
public class Rectangle implements Shape {

    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}
```

```
public class Square implements Shape {

    @Override
    public void draw() {
        System.out.println("Inside Square::draw() method.");
    }
}
```

```
public class Circle implements Shape {

    @Override
    public void draw() {
        System.out.println("Inside Circle::draw() method.");
    }
}
```

实验步骤：图形工厂举例（2）

工厂系：创建工厂接口 ShapeFactory和实现该接口的三个具体工厂实体类。



③ 创建工厂接口ShapeFactory，充当创建者角色；

```
public interface ShapeFactory {

    public abstract Shape createShape();

}
```

④ 创建三个工厂实体类，充当具体创建者角色。

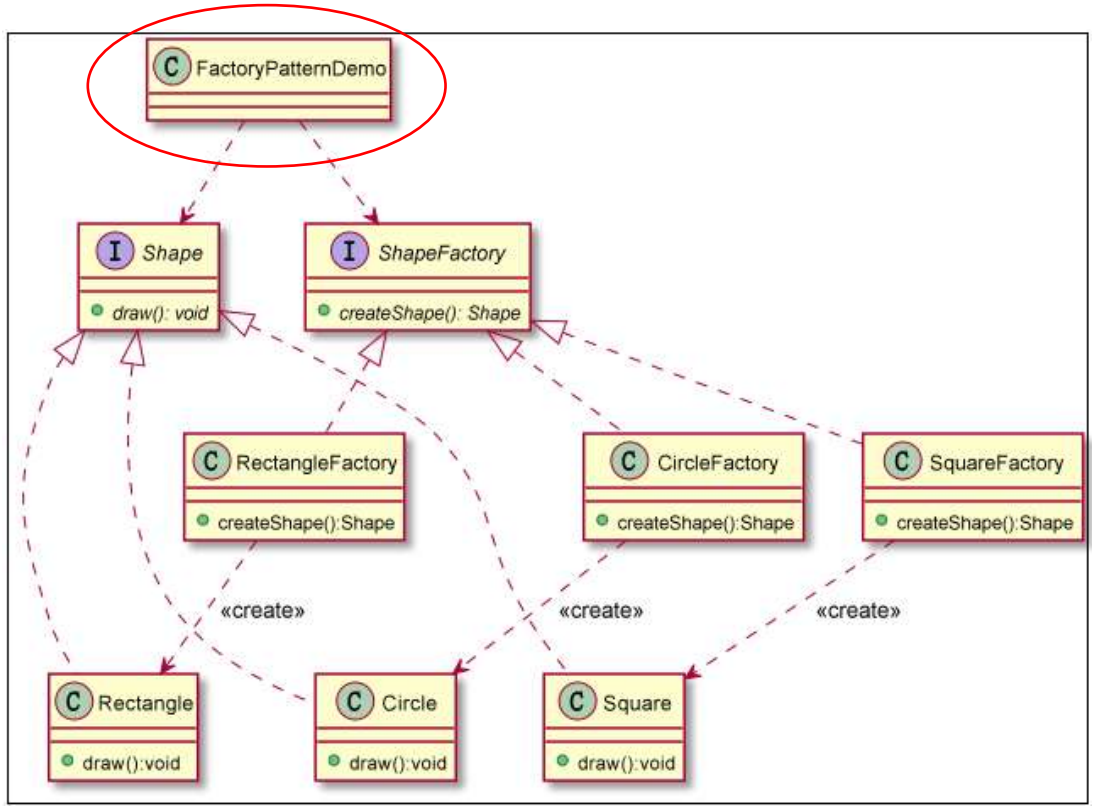
```
public class RectangleFactory implements ShapeFactory {

  @Override
    public Shape createShape() {
        return new Rectangle();
    }
}
```

```
public class SquareFactory implements ShapeFactory {

  @Override
    public Shape createShape() {
        return new Square();
    }
}
```

```
public class CircleFactory implements ShapeFactory {

  @Override
    public Shape createShape() {
        return new Circle();
    }
}
```

实验步骤：图形工厂举例（3）

⑤ 客户端FactoryPatternDemo 类使用 ShapeFactory 来获取不同的Shape对象。

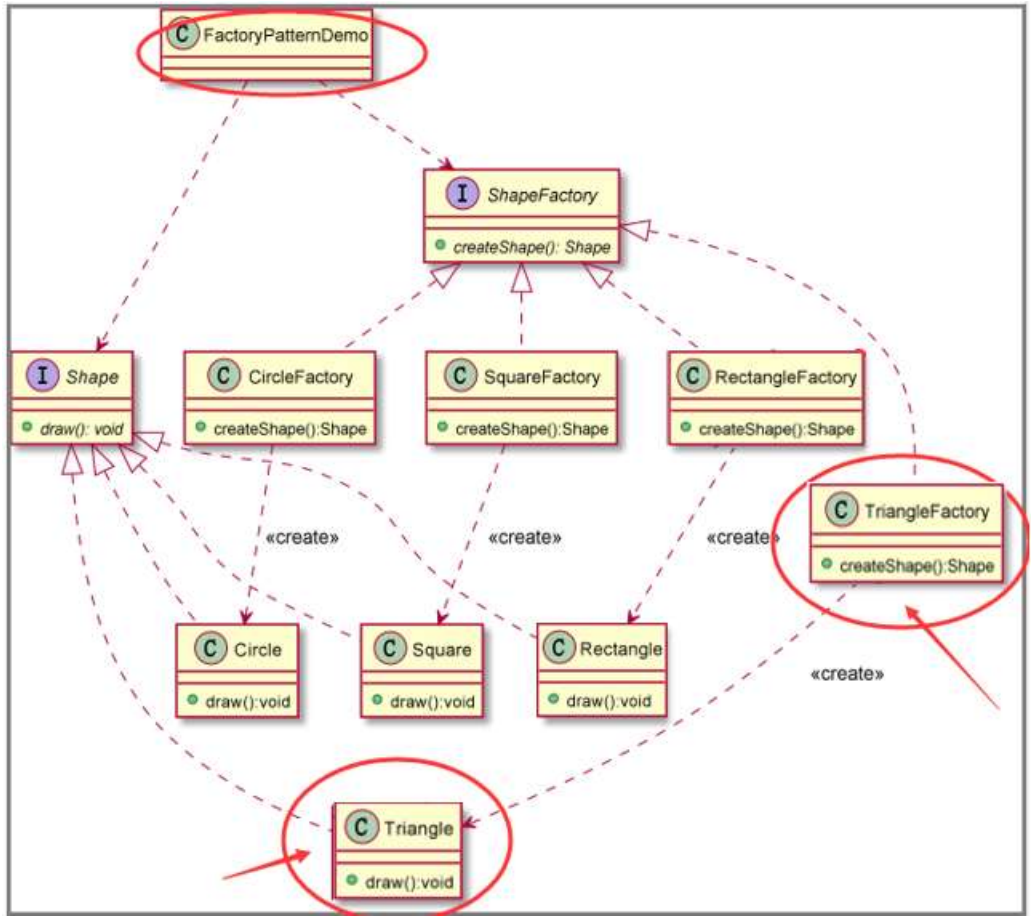



```java
public static void main(String[] args) {

    ShapeFactory shapeFactory ;
    Shape shape;

    //获取 Circle 的对象，并调用它的 draw 方法
    shapeFactory = new CircleFactory();
    shape = shapeFactory.createShape();
    shape.draw();

    //获取 Rectangle 的对象，并调用它的 draw 方法
    shapeFactory = new RectangleFactory();
    shape = shapeFactory.createShape();
    shape.draw();

    //获取 Square 的对象，并调用它的 draw 方法
    shapeFactory = new SquareFactory();
    shape = shapeFactory.createShape();
    shape.draw();

}
```

```
Inside Circle::draw() method.
Inside Rectangle::draw() method.
Inside Square::draw() method.
```

实验步骤：图形工厂举例（4）

请思考：如何添加一个三角形图形？




```java
public class FactoryPatternDemo {

   public static void main(String[] args) {

    ShapeFactory shapeFactory ;
    Shape shape;

    //获取 Circle 的对象，并调用它的 draw 方法
    shapeFactory = new CircleFactory();
    shape = shapeFactory.createShape();
    shape.draw();

    //获取 Rectangle 的对象，并调用它的 draw 方法
    shapeFactory = new RectangleFactory();
    shape = shapeFactory.createShape();
    shape.draw();

    //获取 Square 的对象，并调用它的 draw 方法
    shapeFactory = new SquareFactory();
    shape = shapeFactory.createShape();
    shape.draw();

    //获取 Triangle 的对象，并调用它的 draw 方法
    shapeFactory = new TriangleFactory();
    shape = shapeFactory.createShape();
    shape.draw();

   }
}
```

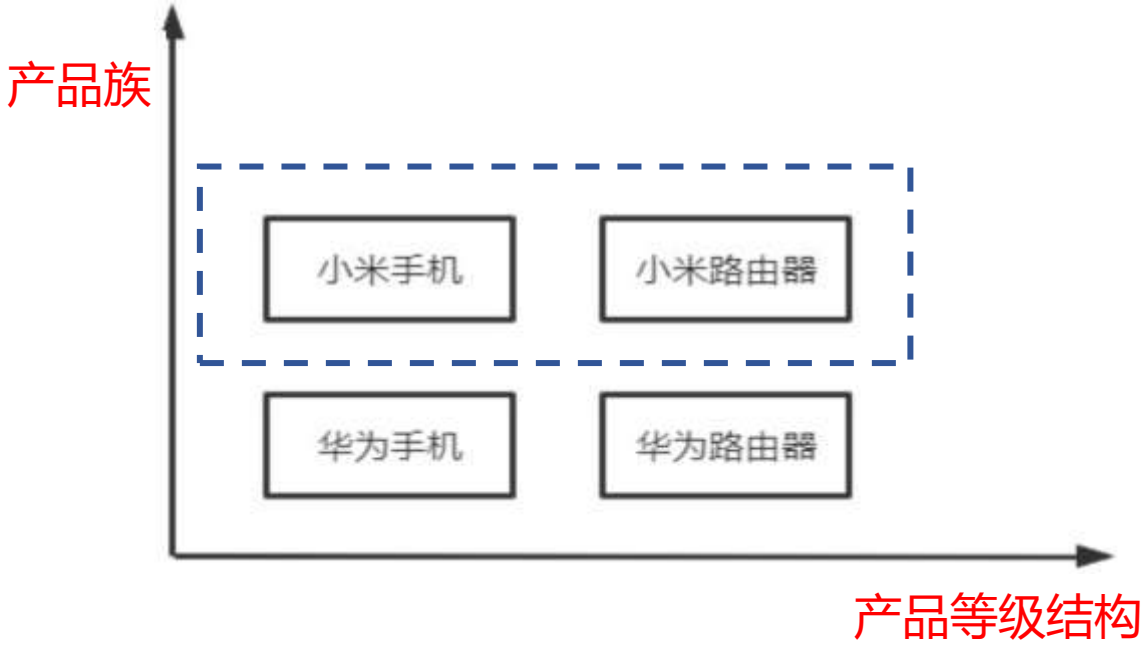
开闭原则

思考题

若采用抽象工厂模式创建敌机和道具是否合适?

A . 合适

B . 不合适



本次迭代开发目标（1）

任务清单
(CheckList)

UML类图设计

① 结合实例，按照规范绘制简单工厂模式与工厂方法模式的结构类图

设计模式应用

② 完成代码重构，使用简单工厂模式实现道具创建，使用工厂方法模式实现敌机创建

敌机生成

③ 界面上可观察到4种敌机（Boss机除外）按周期随机出现

敌机移动

④ 精锐和王牌敌机向屏幕下方移动，且可左右移动

本次迭代开发目标（2）

任务清单
(CheckList)

敌机攻击

⑤ 精锐敌机按设定周期向下直射双排子弹，王牌敌机采用扇形散射弹道，单次同时发射 3 颗子弹

道具生成

⑥ 精锐敌机、王牌敌机被击毁时按概率随机掉落某一种道具；其中精锐敌机可掉落 4 种道具（不含冰冻）中的一种，王牌敌机可掉落5 种道具中的一种

道具效果

⑦ 除加血道具外，其余道具生效时在控制台输出提示信息即可

同学们，请开始实验吧！

THANK YOU